

Capstone Project

Instructions

1. This project must be completed and submitted **individually**.
2. The programming language to be used is **C** only.
3. Use **basic C programming concepts** covered in class. Avoid overly complex code or advanced libraries.
4. Your program should contain **clear comments** explaining important sections of the code.
5. Any work copied directly from AI tools or generated entirely by AI will score **0 marks**.
6. Ensure your code is well arranged, properly indented, and easy to read.
7. Program presentations and demonstrations will be communicated later.
8. Submitting the same code qualifies to a score **0**
9. Clearly show all inputs, processing, and outputs(Screenshots) in your program.
10. Zip your work and upload a Zipped file
11. **All your programs should output your Name and Registration number at the end of the final output**

Question 1 (Owembabazi Tonny, Asingya Kelvin, Honest Fahad, Balikudembe Andrew, Ocen Aaron, Nakirayi Robinah, Kiconco Samuel, Kamusiime Bridget, Baryanengwe Buharl, Illakut Juventine)

In many local communities, land boundary agreements and rental payment records are still managed using paper documents. These documents can easily be lost, damaged, or altered, leading to misunderstandings and disputes between landlords and tenants.

You are required to develop a simple **Land and Rental Records Management System** in C language to help store and manage property and tenant information digitally.

Requirements

Data Structures

Create structures (struct) for the following:

Property

Include:

- Location
- Plot Number
- Owner Name
- Size

Tenant

Include:

- Tenant Name
- National Identification Number (NIN)
- Last Payment Date

Program Functionalities

Your program should be able to:

- Register new land/property details.
- Record rental payments in Uganda Shillings (UGX).
- Search for property ownership using the Plot Number.
- Display stored records clearly to the user.

File Handling

Use file handling to save all records into a file called: **land_records.txt**

Rent Monitoring Feature

Implement a feature that identifies or flags tenants who have not paid rent for more than **3 months** according to the Land Act guidelines.

The program should clearly display the names and details of tenants with overdue rent payments.

Question 2 (Kobugabe Bridget, Nakakawa Maria, Kyalisima Angella, Muhumuza Peter, Musika Alvin, Birimawa Patience, Munguchi Yasir, Niwabikora Promise, Jeero Benon, Kikaziki Palma)

Many Boda-Boda riders belong to small savings groups commonly known as SACCOs. Members contribute daily savings or “stage fees” and may borrow small loans from the group fund.

Managing these records manually can lead to errors, loss of records, and misunderstandings about balances and loan repayments.

You are required to develop a simple **Boda-Boda SACCO Savings Tracker** using the C programming language to help manage members’ savings and loan activities.

Requirements

1. Data Structures

Create a structure (struct) to store rider information. Include the following details:

- Rider Name
- National Identification Number (NIN)
- Stage Name
- Total Savings
- Loan Amount
- Loan Interest

2. Program Functionalities

Your program should allow the SACCO operator to:

- Register new riders.
- Record daily savings deposits.
- Allow riders to borrow money from the SACCO.
- Calculate loan interest on borrowed money.
- Check and display a rider’s savings balance.
- Display all registered riders and their records.

3. Menu System

Use a **switch-case menu** to allow the user to select different program options such as:

1. Deposit Savings
2. Borrow Money
3. Check Balance
4. Display All Records
5. Exit Program

The menu should continue appearing until the user chooses to exit.

4. File Handling

Use file handling to save rider records into a file called: sacco_records.txt

5. Expected Output

Your program should:

- Accept user input correctly.
- Perform calculations accurately.
- Display balances and loan information clearly.
- Show appropriate messages for successful deposits, borrowing, and balance checking.
- Use comments to explain important sections of the code.
- Screenshots of outputs

Question 3: (Mitala Lawrence, Kityo Richard, Muwanguzi Emmanuel, Wasswa Mahad, Mukisa Luke, Mawejje Thomas, Kinene Malcom, Nuwahereza Nathan, Niyonzima Allan, Ssekitoleko Eddie, Itaagi Timothy)

Many farmers in rural areas do not know the current market prices of agricultural products in major markets such as Owino, Nakasero, and Mbarara. Because of this, they often sell their produce at very low prices to middlemen.

You are required to develop a simple **Agro-Price Market Linkage Tool** using the C programming language to help farmers access updated crop prices from different regions of Uganda.

Requirements

1. Data Structures

Create a structure (struct) to store commodity price information. Include the following details:

- Commodity Name
- Market Name
- District/Region
- Price per Kilogram (UGX)
- Date Updated

2. Program Functionalities

Your program should allow the Market Admin to:

- Add new commodity price records.
- Update daily prices of commodities such as:
 - Matooke
 - Beans
 - Maize
- Display all stored commodity prices.
- Search for a commodity by name.

3. Search Feature

Implement a search feature where a farmer enters the name of a crop.

The program should then:

- Display all available prices for that crop from different districts or markets.
- Show the **highest price** recorded.
- Show the **lowest price** recorded.
- Indicate the district or market where each price was found.

4. Menu System

Use a **switch-case menu** with options such as:

1. Add Commodity Prices
2. Update Commodity Prices
3. Search Commodity Prices
4. Display All Records
5. Exit Program

The menu should continue displaying until the user chooses to exit.

5. File Handling

Use file handling to save records into a file called: market_prices.txt

6. Expected Output

Your program should:

- Accept user input correctly.
- Display prices clearly and neatly.
- Correctly identify the highest and lowest prices.
- Show appropriate messages when records are added, updated, or searched.
- Use comments to explain important sections of the code.

Question 4: (Nakitto Jazirah, Sonzi Edirisa, Tebanyang Isaac, Nabulo Vicky, Akumu Mercy, Kyeyune Ashiraf, Muhwezi Tyson, Odong Jacob, Nabulime Shivan, Lugwana Phillip, Among Eunice)

Many electricity consumers find it difficult to understand how prepaid electricity units (Yaka units) are calculated and deducted. Confusion usually arises when VAT, service fees, and different tariff rates for domestic and commercial users are applied.

You are required to develop a simple **Prepaid Meter Simulator** using the C programming language to help users understand how electricity units are loaded and consumed.

Requirements

1. Data Structures

Create a structure (struct) to store meter information. Include the following details:

- Customer Name
- Meter Number
- Customer Type (Domestic or Commercial)
- Current Units
- Token Number
- Amount Paid
- VAT Charged
- Service Fee

2. Program Functionalities

Your program should allow the user to:

- Register a new meter/customer.
- Enter a voucher/token number to purchase electricity units.
- Calculate units received after deducting VAT and service fees.
- Simulate electricity usage by reducing available units over time.
- Display remaining units balance.
- Display customer and transaction details.

3. Unit Calculation

The system should:

- Accept an amount paid in Uganda Shillings (UGX).
- Deduct VAT according to the applicable percentage.
- Deduct the monthly service fee before calculating units.
- Calculate the number of electricity units based on:
 - Domestic tariff rate
 - Commercial tariff rate

- Add purchased units to the customer's meter balance.

4. Usage Simulation

Implement a feature that simulates electricity consumption.

The program should:

- Reduce units based on entered usage values.
- Warn the user when units become low.
- Prevent units from going below zero.

5. Menu System

Use a **switch-case menu** with options such as:

1. Register Customer
2. Buy Units / Enter Token
3. Simulate Power Usage
4. Check Remaining Units
5. Display Customer Details
6. Exit Program

The menu should continue displaying until the user selects the exit option.

6. File Handling

Use file handling to save all customer and meter records into a file called:

yaka_records.txt

7. Expected Output

Your program should:

- Accept and validate user input correctly.
- Display calculations clearly.
- Correctly apply VAT and service fees.
- Accurately calculate remaining units.
- Display warning messages for low balance.
- Use comments to explain important parts of the code.

Question 5: (Muyinda Joseph, Nalwanga Desire, Atwongiire Jonas, Kamuliso Samuel, Okoi Tabica, Tumwebaze Jordan, Okello Emma, Ashaba Kenneth, Ssegawa Tonny, Okello Stephen, Friday Isaac)

Due to the high volume of goods imported through Entebbe and other entry points, many businesses need proper systems to track shipments from international suppliers such as Alibaba and AliExpress to local warehouses in Uganda.

You are required to develop a simple **Import and Logistics Shipment Management System** using the C programming language to help logistics managers track imported goods and calculate shipping costs.

Requirements

1. Data Structures

Create a structure (struct) to store shipment details. Include the following information:

- Tracking ID
- Supplier Name
- Item Description
- Weight (Kilograms)
- Shipping Mode (Air or Sea)
- Shipment Status:
 - In Transit
 - At Customs
 - Arrived
- Shipping Cost
- URA Clearing Fees
- Handling Charges

2. Program Functionalities

Your program should allow the logistics manager to:

- Register new shipment records.
- Update shipment status.
- Search for shipment details using the Tracking ID.
- Display all shipment records.
- Calculate total shipment costs.

3. Shipping Cost Calculation

Create a function that calculates the total shipping cost.

The calculation should include:

- Shipping charges based on the shipment weight.

- URA clearing fees.
- Handling charges.

The program should clearly display:

- Weight of the shipment
- Shipping mode
- Individual charges
- Final total cost

4. Menu System

Use a **switch-case menu** with options such as:

1. Add Shipment Record
2. Update Shipment Status
3. Search Shipment
4. Calculate Shipping Costs
5. Display All Records
6. Exit Program

The menu should continue displaying until the user chooses to exit.

5. File Handling

Use file handling to save shipment records into a binary file called:
shipment_manifest.dat

6. Expected Output

Your program should:

- Accept user input correctly.
- Display shipment information clearly.
- Correctly calculate total shipment costs.
- Show updated shipment statuses properly.
- Use comments to explain important sections of the code.

Question 6: (Muzimba Ian, Mboija Allan, Muyala Carol, Ssemwogerere Ukasha, Odoch Joel, Nakaye Ann, Mwanje Trevor, Behumisa Henry, Opade Paul, Okim Mark, Atuhire Florence)

Mobile money platforms such as MTN MoMo and Airtel Money are widely used for sending money, paying bills, and checking balances. These systems follow a structured transaction process with menu options, PIN validation, and automatic transaction charges.

You are required to develop a simple **Mobile Money Transaction Simulator** using the C programming language to demonstrate how mobile money systems operate.

Requirements

1. Data Structures

Create a structure (struct) to store customer account information. Include the following details:

- Customer Name
- Phone Number
- Account Balance
- PIN
- Transaction Amount
- Transaction Type

2. Program Functionalities

Your program should allow the user to:

- Check account balance.
- Send money to another phone number.
- Pay bills.
- View transaction details.
- Display updated account balances after transactions.

3. Menu System

Implement a **multi-level switch-case menu** with options such as:

Main Menu

1. Check Balance
2. Send Money
3. Pay Bills
4. Exit

Send Money Menu

- Enter receiver phone number
- Enter amount
- Calculate transaction charges
- Confirm transaction

Pay Bills Menu

- Water Bills
- Electricity Bills
- TV Subscription
- Other Bills

The menu should continue displaying until the user selects the exit option.

4. Withdrawal and Transaction Charges

Create functions that automatically calculate transaction charges based on the amount being sent.

The program should:

- Deduct the correct transaction fee.
- Display the fee charged.
- Display the remaining account balance after deduction.

You may use different charge ranges (tiers) depending on the amount entered.

5. Input Validation

Use loops and validation to ensure that:

- The phone number entered contains exactly 10 digits.
- The PIN entered contains exactly 4 digits.
- Invalid entries are rejected until correct input is provided.

The program should display clear error messages for invalid input.

6. File Handling

Use file handling to save transaction records into a file called:
momo_transactions.dat

7. Expected Output

Your program should:

- Accept and validate user input correctly.
- Display menus clearly.
- Correctly calculate transaction charges.
- Update balances accurately after transactions.
- Display appropriate success or error messages.
- Use comments to explain important parts of the code.

Question 7: (Mugalya Ronald, Nabadda Salmah, Kyaligamba Musa, Wambi Benard, Shambe Latif, Muyanja Clever, Owori James, Kobugabe Bridget, Ainembabzi Violah, Rwotomiyo Maxwell, Namulindwa Angel)

The renewable energy sector is growing rapidly, leading to increased demand for products such as solar panels, inverters, and lithium batteries. Businesses dealing in these products require proper inventory systems to monitor stock levels and avoid shortages.

You are required to develop a simple **Solar Energy Inventory Management System** using the C programming language to help manage stock records and sales of renewable energy products.

Requirements

1. Data Structures

Create a structure (struct) to store product information. Include the following details:

- Product Name
- Brand Name
- Serial Number
- Quantity in Stock
- Unit Price
- Product Type:
 - Solar Panel
 - Inverter
 - Lithium Battery

2. Program Functionalities

Your program should allow the user to:

- Add new products to the inventory.
- Record product sales.
- Update stock quantities after sales.
- Search for products.
- Display all available inventory records.

3. Low Stock Alert System

Implement a feature that automatically checks stock levels whenever a sale is recorded.

The program should:

- Reduce the available quantity after each sale.
- Display a **Low Stock Alert** if the remaining quantity falls below a specified limit (for example, 5 units).
- Prevent sales that exceed the available stock quantity.

4. Search Feature

Implement a search algorithm that allows the user to search for products using:

- Serial Number
- Brand Name

The program should display matching product details clearly.

5. Menu System

Use a **switch-case menu** with options such as:

1. Add Product
2. Record Sale
3. Search Product
4. Display All Products
5. Exit Program

The menu should continue displaying until the user selects the exit option.

6. File Handling

Use file handling to save inventory records into a file called:
solar_inventory.dat

7. Expected Output

Your program should:

- Accept user input correctly.
- Display inventory records clearly.
- Correctly update stock quantities after sales.
- Display low stock warnings appropriately.
- Successfully search for products.
- Use comments to explain important sections of the code.